Step 4 — A Verifier Agent That Catches the Worker's Bad Answers — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — What the Verifier sees, and what it returns
- Hint 2 — The plan, and where each piece lives

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — What the Verifier sees, and what it returns

The Verifier is a second agent with one job: judge the Worker's answer. It must **not** share the Worker's prompt — a checker that thinks like the writer tends to agree with it.

It needs three things:

- the original question;
- the Worker's final answer;
- what the tools **actually returned** — otherwise it can only judge whether the answer *sounds* right.

Build it as a Doc05 chain: a prompt with `{question}`, `{answer}` and `{evidence}`, piped into the model.

Make the verdict a checked object, not a word in a string:

```
class VerifierVerdict(BaseModel):
    approved: bool
    reason: str
```

`prompt | model.with_structured_output(VerifierVerdict)` returns a real `VerifierVerdict` (Doc05's structured parser exercise).

For the evidence, write `collect_evidence(steps) -> str`: one line per step whose `error` is `None` — `"- get_weather returned: ..."`. Skip the failed attempts: the Verifier should judge the answer, not how many retries it took. With no successful steps, return `"(no tools were used)"`.

> Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and where each piece lives

```
build_verifier_chain(model):
    prompt from verifier.txt | model with structured output

verify(chain, question, result):
    verdict = chain.invoke({question, answer, evidence})
    log it, return it

main loop:
    result = run_worker(...)
    verdict = verify(...)
    print "Answer (verified)" or "Answer (NOT verified — reason)"
```

Put the handoff in `services/research_service.py`:

```
answer_task(model, verifier_chain, task, max_iterations):
    result = run_worker(model, task, max_iterations)
    verdict = verify(verifier_chain, task, result)
    if not verdict.approved:
        log a warning
        retry_task = task + the Verifier's reason + "check again"
        result = run_worker(model, retry_task, max_iterations)
        verdict = verify(verifier_chain, task, result)   # original task
    return result, verdict
```

Only **one** retry — enough to fix a slip, never an endless loop.

The most important test: build an `AgentResult` by hand whose steps say `64.4` but whose answer says `90°F`, and check the Verifier **rejects** it. Then the same steps with `64.4°F` must be approved. A Verifier that never rejects anything hasn't been tested.

Last, `data/sample/tasks.json` and `scripts/compare_with_tools.py`: for each task, print Step 1's from-memory answer next to `answer_task()`'s verified one.

Full solution: Show me the solution